Lecture plan · for the lecturer

Planning and tool use with language models

AI agents · University · Lecture 2

Time	Materials
60 min	Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- Plan with A*.
- Build ReAct agents.
- Ground answers in tools.

Activity: Search, then act

1. Warm-up (5 min): ما الخطأ؟
2. Trace (20 min): Pairs trace A* on a six-node map with the open list on paper.
3. Debug (15 min): Find the missing tool call in a ReAct transcript and rewrite the step.
4. Compare (15 min): List what A* guarantees that the agent does not.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- When is A* optimal?
- Why did the agent skip the tool?
- How will you grade transcripts?

Exit ticket

- Write one instruction that grounds an agent’s answers.

Try it in your next lecture

Trace A* on a six-node campus map on the board, then show a ReAct transcript with one missing tool call and ask pairs to find the step where the agent should have acted.

Seminar sheet · for students

Planning and tool use with language models

AI agents · University · Lecture 2

Name _____ Class _____ Date _____

1 **Which change most directly stops the agent inventing a room?**

◯ A bigger model ◯ Require and check a tool result ◯ A longer answer

2 **Expand A*.**

Node X has g = 4 and h = 3; node Y has g = 2 and h = 6. A* expands _____ first because its f is _____.

Word bank: X · Y · 8 · 7

3 **Which reduce hallucinated answers in a tool-using agent? Choose all that apply.**

☐ Clear, specific tool descriptions ☐ Instructions to answer only from tool results

☐ Validating answers against tool output ☐ Removing the step limit

4 **A ReAct agent, like A* with an admissible heuristic, guarantees an optimal plan.**

◯ True ◯ False

5 **Order one ReAct step.**

☐ Answer or next thought

☐ Observation: read the result

☐ Action: call a tool

☐ Thought: what is needed

✎ **Do your students learn classical search before they build language-model agents? What would they miss without it?**